

**UNIVERSIDAD NACIONAL
DE LA PATAGONIA AUSTRAL (UNPA)**

TRABAJO DE CAMPO 2022

Agencia de castings

ALUMNA

Mendez Garai Florencia Estefania

MATERIA

Bases de Datos

PROFESORES

Ing. Altamirano Walter,

Lic. Bianchiotti Franco

Avance N°2

ÍNDICE

1.Introducción	2
2.Propósito	2
a)Objetivo general	2
b)Objetivo específico	2
3.Definición del caso de estudio	3
4.Alcance	3
5.Análisis y recopilación de requisitos	4
a)Descripción	4
b)Requerimientos de usuario	4
c)Requerimientos funcionales	5
d)Requerimientos no funcionales	6
6.Diseño Conceptual	7
a)Descripción de entidades y atributos	7
b)Relaciones	9
c)Participación y cardinalidad	10
d)Supuestos asumidos para la resolución del problema	13
e)Restricciones	13
7. Elección del SGBD	14
a)Descripción	14
b)Justificación	14
8.Diseño Lógico	14
a)Descripción	14
b)Diseño Lógico Estándar	15
1.TRANSFORMACIÓN MER-MR	15
2.Grafo relacional	19
c)Normalización	20
d)Diseño lógico específico	23
9.Implementación	24
10.Conclusión	38
11.Bibliografía	39

1. Introducción

En este documento se llevará a cabo el desarrollo del trabajo académico integrador, que forma parte del examen final de la asignatura Bases De Datos correspondiente a la carrera Analista De Sistemas en la UNPA- UARG. El fin de este documento es poder demostrar cómo se diseña e implementa una base de datos, trabajando con un caso ficticio de una agencia de castings, que se dedica a buscar modelos y actores/actrices para sus clientes.

En primer lugar, para poder continuar con el desarrollo de este trabajo, se debe definir ¿Qué es una Base de Datos?

Una **Base de Datos** es una *colección de datos relacionados*. Con la palabra datos nos referimos a *los hechos (datos) conocidos que se pueden grabar y que tienen un significado implícito*. Sin embargo, la definición de una base de datos va más allá de ser un **conjunto de datos que poseen una relación**, sino que cuando se habla de una **Base de Datos**, se consideran tres propiedades implícitas que esta posee, la primera de ellas es la *representación algún aspecto del mundo real*, lo que en ocasiones se denomina mini mundo o universo de discurso, la segunda es ser una *colección de datos lógicamente coherente* con algún tipo de significado inherente y la última es el *diseño, construcción y carga con datos sobre la misma para un propósito específico*.

Luego de definir qué es una Base de Datos, lo que interesa saber es, ¿cómo se diseña e implementa una Base de Datos? ¿qué pasos se deben de seguir?

Existen diferentes metodologías que podrían adoptarse para hacerlo, pero en el desarrollo de este trabajo utilizaremos una metodología de diseño de Base De Datos relacionales, la cual se divide en seis etapas, la primera es **el análisis y recopilación de requisitos**, la segunda **el diseño conceptual (que utiliza el modelo ER y extensiones)**, la tercera **la elección del SGBD**, la cuarta **el diseño lógico**, la quinta **el diseño físico** y por último la **implementación**.

2. Propósito

a) Objetivo general

La finalidad de este proyecto es exponer los conceptos y prácticas desarrollados durante la cursada mediante la definición, implementación y manipulación del esquema de datos de una aplicación, de acuerdo a los requerimientos que se han recopilado en la siguiente sección.

b) Objetivo específico

Realizar el diseño e implementación de una Base de Datos eficiente, bien diseñada y fácil de mantener, que cumpla con lo esperado por parte de la agencia de castings.

3. Definición del caso de estudio

Una Agencia de Castings dedicada a buscar modelos y actores/actrices para sus clientes desea diseñar una base de datos que cumpla con determinadas condiciones.

Las condiciones son las siguientes:

- Un casting se identifica por un código se caracteriza por un nombre y una fecha de contratación.
- Un casting es contratado por un único cliente, identificado por un código de cliente, aunque un cliente puede tener contratados varios castings. Un casting tiene además un presupuesto y es dirigido por un agente identificado por su número de legajo del mismo
- Un casting se estructura en varias fases, identificadas dentro de cada casting por un número en secuencia y a su vez, cada fase se descompone en varias pruebas individuales identificadas por un número de prueba individual dentro de cada fase. Cada fase tiene una fecha de inicio
- De cada prueba individual se guarda la fecha de realización y la hora de inicio y de finalización, así como la sala en la que se realiza.
- En una sala solo se realizará una prueba en una determinada fecha entre una hora de inicio y una hora de fin.
- A cada casting se le asigna uno o varios perfiles identificados por un código de perfil y con una serie de atributos que denominaremos Atributos de Perfil.
- Los candidatos de la agencia se identifican por un código y tienen además un nombre y una dirección. Cada candidato tiene un único perfil, pero un perfil puede corresponder a varios candidatos.
- Cada candidato que encaje con el perfil de un casting realizará una prueba individual y obtendrá un resultado que puede ser “apto” o “no apto”. Un candidato solo puede realizar pruebas de castings compatibles con su perfil.
- Un candidato puede someterse como máximo a una prueba individual dentro de cada fase de un casting.
- En cada prueba individual de una fase solo la participa un único candidato. Cada prueba en la que participa un candidato pertenece sólo a una fase de un casting.
- Un candidato no podrá realizar una prueba individual de una fase si en la fase anterior realizó una prueba cuyo resultado fue “no apto”.

4. Alcance

a) Descripción del alcance del proyecto

El alcance de este proyecto es realizar el diseño de una base de datos relacional que se encargue de la selección de los actores/actrices para la agencia de castings de este caso ficticio.

5. Análisis y recopilación de requisitos

a) Descripción

Como se decía en un principio, el análisis y recopilación de requisitos es la primera etapa del proceso de diseño de una Base De Datos, pero ¿Qué es lo que se hace en esta etapa?

En el análisis de requisitos se responde a la pregunta: ¿qué representar? Para ello hay que estudiar las reglas de la empresa (del negocio) y los diferentes niveles de la organización para elaborar una descripción de la organización, utilizando el lenguaje natural que lleva al usuario a establecer sin formalismo cuál es el problema a resolver. El resultado de esto es un conjunto por escrito de los requisitos del usuario, que se plasman en un formulario lo más detallado y completo posible. En paralelo al estudio de estos requisitos, se especifican los *requisitos funcionales* de la aplicación, que consisten en las operaciones (o transacciones) definidas por el usuario que se aplicarán a la base de datos, incluyendo las recuperaciones y las actualizaciones.

En este caso, el análisis de los requisitos se ha realizado de acuerdo a la información proporcionada por la agencia de castings, estableciendo los datos que deben almacenarse en la base de datos y las relaciones que existen entre ellos.

b) Requerimientos de usuario

RU01: Un casting se identifica por un código se caracteriza por un nombre y una fecha de contratación.

RU02: Un casting es contratado por un único cliente, identificado por un código de cliente, aunque un cliente puede tener contratados varios castings.

RU03: Un casting tiene además un presupuesto y es dirigido por un agente identificado por su número de legajo del mismo.

RU04: Un casting se estructura en varias fases, identificadas dentro de cada casting por un número en secuencia y a su vez, cada fase se descompone en varias pruebas individuales identificadas por un número de prueba individual dentro de cada fase. Cada fase tiene una fecha de inicio.

RU05: De cada prueba individual se guarda la fecha de realización y la hora de inicio y de finalización, así como la sala en la que se realiza.

RU06: En una sala solo se realizará una prueba en una determinada fecha entre una hora de inicio y una hora de fin.

RU07: A cada casting se le asigna uno o varios perfiles identificados por un código de perfil y con una serie de atributos que denominaremos Atributos de Perfil.

RU08: Los candidatos de la agencia se identifican por un código y tienen además un nombre y una dirección.

RU09: Cada candidato tiene un único perfil, pero un perfil puede corresponder a varios candidatos.

RU10: Cada candidato que encaje con el perfil de un casting realizará una prueba individual y obtendrá un resultado que puede ser “apto” o “no apto”.

RU11: Un candidato solo puede realizar pruebas de castings compatibles con su perfil.

RU12: Un candidato puede someterse como máximo a una prueba individual dentro de cada fase de un casting.

RU13: En cada prueba individual de una fase solo la participa un único candidato. Cada prueba en la que participa un candidato pertenece sólo a una fase de un casting.

RU14: Un candidato no podrá realizar una prueba individual de una fase si en la fase anterior realizó una prueba cuyo resultado fue “no apto”.

c) Requerimientos funcionales

RF01: El sistema deberá almacenar para cada casting un código, un nombre, presupuesto y una fecha de contratación

RF02: El sistema deberá almacenar un único cliente para cada casting, pero puede almacenar varios castings para un cliente

RF03: El sistema deberá almacenar para cada cliente un código de cliente

RF04: El sistema deberá almacenar para cada agente un numero de legajo

RF05: Cada casting es dirigido por un agente

RF06: El sistema deberá almacenar para cada fase de un casting, un número de secuencia y una fecha de inicio

RF07: El sistema deberá almacenar para cada prueba individual que forma parte de un casting, un número de prueba, fecha de realización, la hora de inicio, hora de finalización y sala

RF08: El sistema tendrá que almacenar para cada sala correspondiente a un casting, una prueba individual en una determinada fecha entre una hora de inicio y una hora de fin.

RF09: El sistema almacenara cada perfil de un casting con un código y atributos

RF10: Para cada casting, el sistema deberá almacenar uno o varios perfiles

RF11: El sistema guardara para cada candidato de la agencia de casting un código, un nombre y una dirección

RF12: El sistema tendrá que guardar para cada candidato un único perfil, pero podrá guardar un perfil para varios candidatos

RF13: El sistema deberá almacenar un resultado (apto, no apto) para cada candidato que realice una prueba individual

RF14: Para cada candidato, el sistema guardara como máximo a una prueba individual dentro de cada fase de un casting.

RF15: Para cada prueba individual perteneciente a una fase de un casting, el sistema solo guardara un único candidato

RF16: Para cada prueba individual que realiza un candidato durante un casting, el sistema guardara solo una fase

RF17: El sistema deberá impedir que un candidato realice una prueba individual de una fase si en la fase anterior obtuvo un resultado “no apto”

d) Requerimientos no funcionales

RNF01: El sistema deberá garantizar que los datos de los candidatos estén protegidos mediante autenticación y control de acceso

RNF02: El sistema deberá permitir el acceso simultáneo de múltiples usuarios sin afectar el rendimiento

RNF03: El sistema deberá responder a las consultas en menos de 2 segundos

RNF04: La base de datos deberá ser compatible con el SGBD MySQL 8.0

RNF05: El sistema deberá poder ser ejecutado en cualquier sistema operativo compatible con MySQL Workbench

6. Diseño Conceptual

Una vez terminada la etapa de análisis y recopilación de requisitos, se procede a la etapa de diseño conceptual. El diseño conceptual permite realizar un *esquema conceptual* de alto nivel e independiente de detalles de implementación partiendo de la especificación de requisitos obtenida en la etapa anterior. Para obtener el esquema conceptual se hace uso del *Modelo Entidad Relación*, este modelo proporciona una forma representar los datos que son guardados en una base de datos y las relaciones que existen entre ellos. Este modelo trabaja con los conceptos *entidad*, *atributo* y *relaciones*

a) Descripción de entidades y atributos

Se puede definir una **entidad** como cualquier **objeto (real o abstracto)** que existe en la realidad y acerca del cual queremos almacenar información en la base de datos.

Las características o propiedades de las entidades que describen aspectos específicos de ellas se las llama **atributo**. Los atributos se clasifican en **simples, compuestos y derivados**, así como **monovalorados o multivalorados**.

- ✓ Si un atributo no puede dividirse se lo denomina *simple*, en cambio, si puede dividirse en otros se lo llama *compuesto*.
- ✓ Si un atributo deriva de otro atributo se lo denomina *derivado*
- ✓ Si un atributo contiene solo un valor se lo llama *monovalorado*
- ✓ Si un atributo tiene más de un valor se lo denomina *multivalorado*

Además de esta clasificación encontramos otro tipo de atributos llamados atributos *claves*, los atributos claves tienen **valor distinto** para cada instancia de un tipo de entidad, una clave identifica de forma única cada entidad concreta.

Las entidades se pueden clasificar en:

- **Entidades fuertes (o regulares):** Poseen una clave primaria propia que las identifica de forma única.
- **Entidades débiles:** No tienen un atributo clave propio y dependen de una entidad fuerte para su **existencia e identificación**.
 - **Dependencia en existencia:** La existencia de una entidad débil depende de una entidad fuerte (etiqueta "E" en el MER). Si la entidad fuerte se elimina, las instancias de la entidad débil asociadas también se eliminan.

- **Dependencia en identificación:** Las entidades débiles no se identifican por sí solas; su clave completa se forma combinando la clave primaria de la entidad fuerte con una **clave parcial** (o discriminante) de la entidad débil.
- **Relación identificadora:** Conecta la entidad débil con la entidad fuerte, definiendo su dependencia.

Las entidades y atributos identificados en el caso ficticio son los siguientes:

CASTING

Entidad	Atributos	Tipo	Descripción	Clave
CASTING	cod_casting	Simple		Si
	nombre	Simple		No
	fecha_contratacion	Simple		No
	presupuesto	Simple		No

Tabla 1 – entidad casting

CLIENTE

Entidad	Atributos	Tipo	Descripción	Clave
CLIENTE	cod_cliente	Simple		Si
	nombre	Simple		no
	teléfono	Simple		no
	email	Simple		no
	dirección	Compuesto	nro-calle	no

Tabla 2 – entidad cliente

AGENTE

Entidad	Atributos	Tipo	Descripción	Clave
AGENTE	nro_legajo	Simple		Si
	nombre	Simple		no
	teléfono	Simple		no
	dirección	Compuesto	nro-calle	no

Tabla 3 – entidad agente

FASE

Entidad	Atributos	Tipo	Descripción	Clave
FASE	nro_secuencia	Simple		Si
	fecha_inicio	Simple		no

Tabla 4 – entidad fase

PRUEBA INDIVIDUAL

Entidad	Atributos	Tipo	Descripción	Clave
PRUEBA_INDIVIDUAL	nro_prueba	Simple		Si
	fecha_realizacion	Simple		no
	hora_inicio	Simple		no
	hora_finalizacion	Simple		no
	sala	Simple		no
	resultado	Simple		no

Tabla 5 – entidad prueba_individual

PERFIL

Entidad	Atributos	Tipo	Descripción	Clave
PERFIL	cod_perfil	Simple		Si
	atributos	Simple		no

Tabla 6 – entidad perfil

CANDIDATO

Entidad	Atributos	Tipo	Descripción	Clave
CANDIDATO	código	Simple		si
	nombre	Simple		no
	dirección	Compuesto	nro - calle	no

Tabla 7 – entidad candidato

b) Relaciones

Cuando hablamos de *relación* nos referimos a aquella asociación, [vínculo](#) o correspondencia [entre instancias de entidades](#) relacionadas de alguna manera en el “mundo real”.

	CASTING	CLIENTE	AGENTE	FASE	PRUEBA_ INDIVIDUAL	PERFIL	CANDIDATO
CASTING		X	X	X		X	
CLIENTE	X						
AGENTE	X						
FASE	X				X		
PRUEBA_ INDIVIDUAL				X			X
PERFIL	X						X
CANDIDATO					X	X	

Tabla 8 – relaciones

c) Participación y cardinalidad

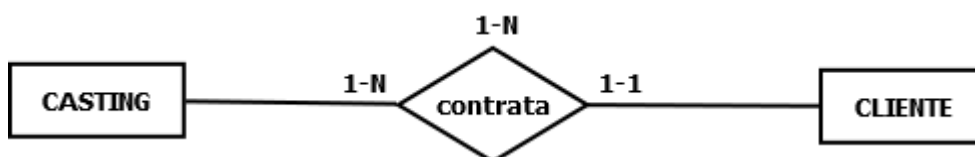
La *participación* es el número máximo de veces que una entidad puede participar en una relación. Los valores que puede tomar la participación entre entidades: (0,1). Mínimo cero, máximo 1, (1,1). Mínimo 1 máximo 1, (0, n). Como mínimo cero, como máximo muchas veces. (1, n). Uno como mínimo, máximo muchas veces.

La *cardinalidad* es el número máximo de instancias de tipo de relación en las que puede participar una misma instancia de tipo de entidad. Gráficamente se representa con una etiqueta en la línea que une entidad y relación. Existe tres razones de cardinalidad más comunes: 1:1 (“uno a uno”), 1: N (“uno a muchos”) y M: N (“muchos a muchos”).

1. Relación entidad CASTING – contrata – entidad CLIENTE

Participación: Relación binaria (GRADO 2)

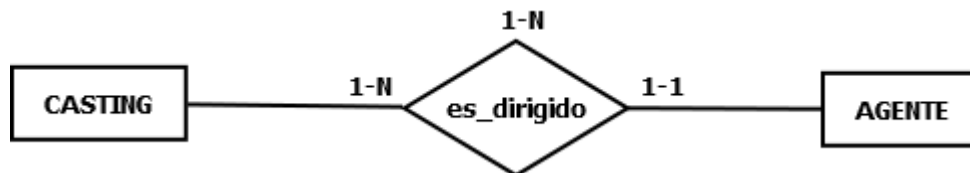
Cardinalidad: Un casting es contratado por un único cliente y un cliente puede tener contratados varios castings

Figura 1 – relación casting *contrata* cliente

2. Relación entidad CASTING – es_dirigido – entidad AGENTE

Participación: Relación binaria (GRADO 2)

Cardinalidad: Un casting es dirigido por un agente y un agente puede dirigir varios castings

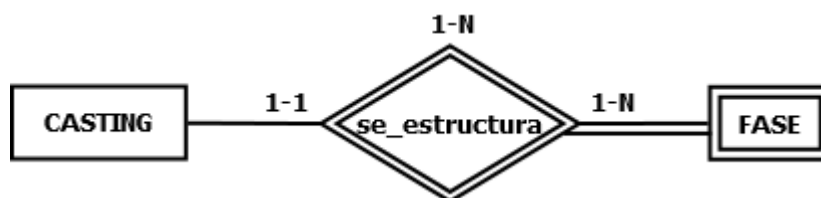


*Figura 2 – relación casting **es dirigido** agente*

3. Relación entidad CASTING – se estructura – entidad FASE

Participación: Relación binaria (GRADO 2)

Cardinalidad: Un casting se estructura en varias fases y cada fase pertenece a un único casting

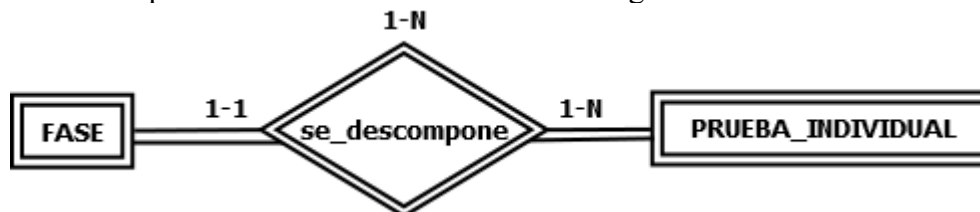


*Figura 3 – relación casting **se estructura** fase*

4. Relación entidad FASE - se descompone – entidad PRUEBA INDIVIDUAL

Participación: Relación binaria (GRADO 2)

Cardinalidad: Cada fase se descompone en varias pruebas individuales y cada prueba individual pertenece sólo a una fase de un casting.



*Figura 4 – relación fase **se descompone** prueba_individual*

5. Relación entidad CASTING – se asigna – entidad PERFIL

Participación: Relación binaria (GRADO 2)

Cardinalidad: A cada casting se le asigna uno o varios perfiles y un perfil puede ser asignado para varios castings

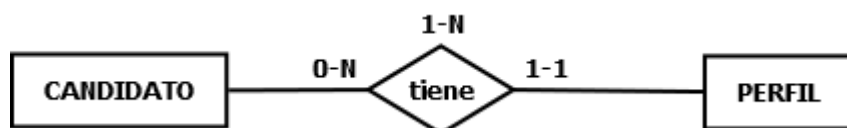


*Figura 5 – relación casting **se asigna** perfil*

6. Relación candidato – tiene – perfil

Participación: Relación binaria (GRADO 2)

Cardinalidad: Cada candidato tiene un único perfil, pero un perfil puede corresponder a varios candidatos.

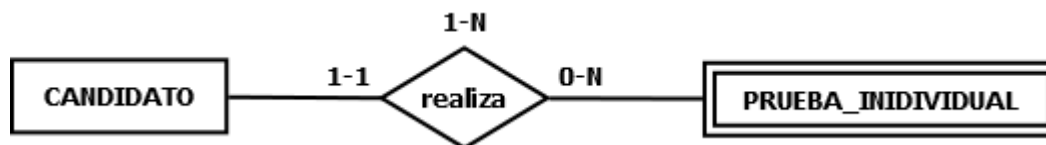


*Figura 6 – relación candidato **tiene** perfil*

7. Relación candidato – realiza – prueba individual

Participación: Relación binaria (GRADO 2)

Cardinalidad: Un candidato realiza de cero a muchas pruebas individuales por fase por casting y una prueba individual es realizada por un único candidato.



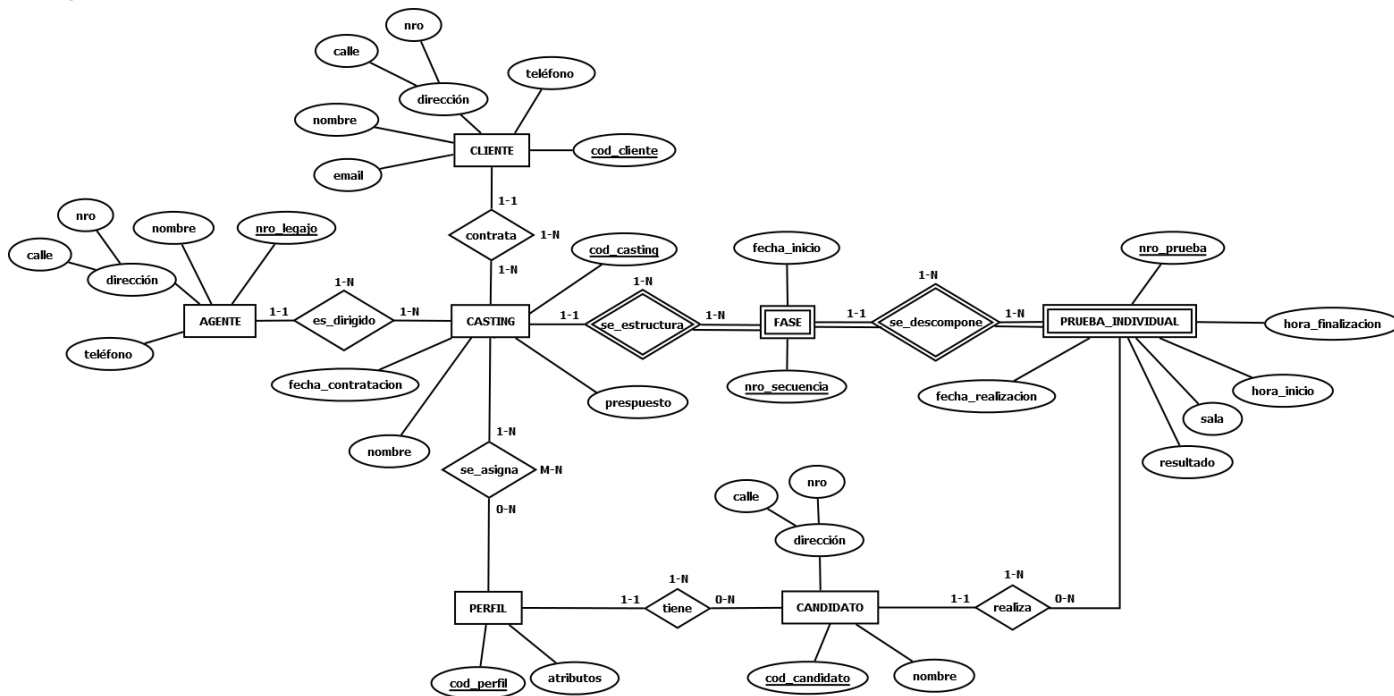
*Figura 7 – relación candidato **realiza** prueba individual*

d) Supuestos asumidos para la resolución del problema

1. un agente puede dirigir varios castings
2. un perfil puede ser asignado para varios castings
3. Un candidato puede no realizar ninguna prueba individual, por lo que no necesariamente participa en ellas. En cambio, cada prueba individual debe estar asignada a un candidato.

e) Restricciones

1. Un candidato solo puede realizar pruebas de castings compatibles con su perfil.
2. Un candidato no podrá realizar una prueba individual de una fase si en la fase anterior realizó una prueba cuyo resultado fue “no apto”.

Diagrama Entidad-Relación**Figura 8 - versión final del Diagrama Entidad Relación (DER)**

7. Elección del SGBD

a) Descripción

Un Sistema de Gestión de Bases de Datos (SGBD) es un *conjunto de programas diseñado para funcionar como interfaz entre las bases de datos y las aplicaciones que las utilizan*, permitiendo a los usuarios **crear y mantener bases de datos** de forma eficiente. Su objetivo es almacenar y recuperar información, facilitando la definición, construcción y manipulación de los datos.

b) Justificación

Para este trabajo final se eligió trabajar con **MySQL Server** como sistema de gestión de bases de datos (SGBD), acompañado por la herramienta gráfica **MySQL Workbench**. Esta combinación permite implementar el modelo relacional de manera clara, manteniendo la estructura lógica definida en el diseño previo.

MySQL Server proporciona soporte para elementos clave del diseño lógico, como claves primarias, claves foráneas, restricciones de integridad y normalización. A su vez, MySQL Workbench facilita la creación y visualización del esquema mediante diagramas, lo que agiliza el desarrollo y mejora la comprensión del sistema en su conjunto.

8. Diseño Lógico

a) Descripción

El diseño lógico es una etapa clave en el proceso de desarrollo de bases de datos, donde se transforma el modelo conceptual (usualmente expresado con el modelo Entidad/Interrelación - E/R) en un esquema lógico, de acuerdo al modelo relacional. Esta conversión se realiza sin tener en cuenta aún las características específicas del sistema gestor de bases de datos (SGBD) que se utilizará más adelante.

Su objetivo principal es estructurar los datos de forma clara, sin redundancias y con reglas precisas que aseguren la coherencia y la integridad. A diferencia del modelo conceptual, que representa la información en términos cercanos al mundo real, el modelo lógico se enfoca en cómo organizar esa información en tablas, atributos y claves.

Este diseño permite definir con exactitud qué datos se almacenarán, cómo se relacionan entre sí y qué restricciones deben cumplirse, pero sin considerar aún aspectos físicos como el almacenamiento o los índices. El resultado final de esta etapa es un esquema lógico que sirve de base para la implementación y que es independiente del producto comercial que se elija para gestionar la base.

b) Diseño Lógico Estándar

El **Diseño Lógico Estándar (DLS)** consiste en transformar el **modelo conceptual** (formulado en el modelo Entidad-Relación Extendido – MERE) en un **esquema lógico estándar (ELS)**, utilizando un **modelo lógico de datos estándar (MLS)**, como el modelo relacional. Esta transformación se realiza sin depender de un SGBD específico, lo que permite mantener la portabilidad y flexibilidad del diseño.

1. TRANSFORMACIÓN MER-MR

La transformación del modelo Entidad-Relación (MER) al modelo Relacional (MR) permite llevar un diseño conceptual a un diseño lógico relacional. Esta transformación se basa en reglas específicas según el tipo de relación entre entidades:

Relaciones 1:1

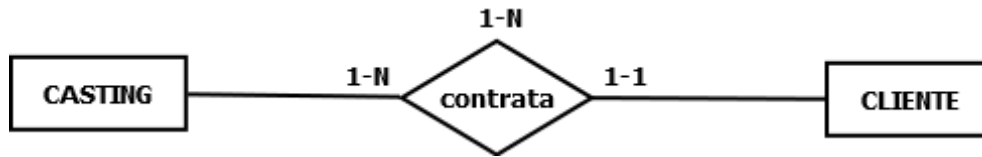
- ❖ **(1,1) con (1,1)**: Se fusionan ambas entidades en una única tabla. Se elige como clave primaria una de las claves originales.
- ❖ **(1,1) con (0,1)**: Se generan dos tablas. La clave primaria de la entidad con participación (1,1) se convierte en clave foránea en la otra.
- ❖ **(0,1) con (0,1)**: Se crean tres tablas: una para cada entidad y otra para la relación, cuya clave primaria es la combinación de las claves de ambas entidades (también claves foráneas).

Relaciones 1: N

- ❖ **Participación mínima (1,1)** del lado “1”: No se crea tabla para la relación. Se propaga la clave del lado “1” al lado “N” como clave foránea. Los atributos de la relación se incorporan a la tabla del lado “N”.
- ❖ **Participación mínima (0,1)** del lado “1”: Se generan tres tablas. La relación se transforma en tabla. Su clave primaria es la de la entidad del lado “N” (también foránea). Además, se incluye la clave del lado “1” como foránea y los atributos propios de la relación.

Relaciones N:M

- ❖ Se crean tres tablas. La tabla de la relación tiene como clave primaria la concatenación de las claves de ambas entidades (también claves foráneas). Los atributos de la relación se incluyen en esta tabla intermedia.

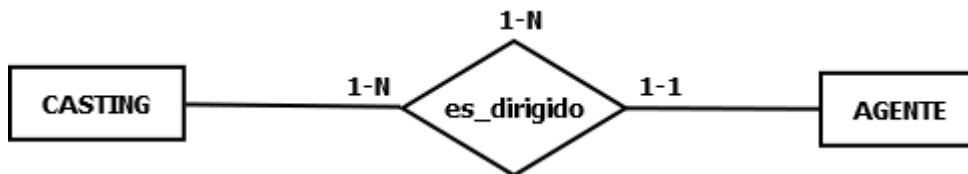
1. Relación entidad CASTING – contrata – entidad CLIENTE**Figura 9 – relación casting *contrata* cliente transformación MER A MR**

Al tener una cardinalidad (1,1) y (1, N), esta relación no se transforma en una tabla independiente. La clave primaria de cliente(cod_cliente) se propaga a casting como clave foránea y los atributos de la relación también se añaden a casting si es que los hubiera.

Como **resultado**, las tablas quedan de la siguiente forma:

CASTING (cod_casting, presupuesto, nombre, fecha_contratacion, *cod_cliente*)

CLIENTE (cod_cliente, nombre, teléfono, dirección, email)

2. Relación entidad CASTING – es dirigido – entidad AGENTE**Figura 10 – relación casting *es dirigido* cliente transformación MER A MR**

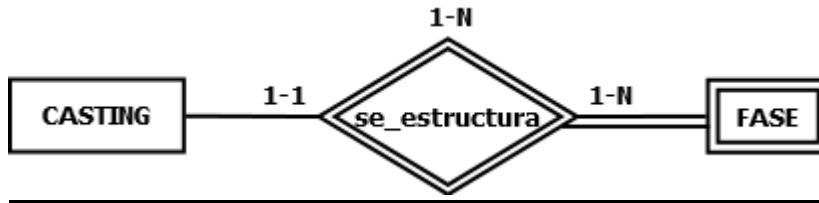
Con una cardinalidad (1,1) y (1, N), esta relación tampoco se convierte en tabla. La clave primaria de agente (nro_legajo) se transfiere a casting como clave foránea.

Como **resultado**, las tablas quedan de la siguiente forma:

CASTING (cod_casting, presupuesto, nombre, fecha_contratacion, *cod_cliente*, *nro_legajo*)

AGENTE (nro_legajo, nombre, teléfono, dirección)

3. Relación entidad CASTING – se_estructura – entidad FASE



*Figura 11 – relación casting **se estructura** fase transformación MER A MR*

Debido a que la entidad fase es una entidad débil por dependencia en identificación respecto de casting, y la relación se_estructura tiene cardinalidad (1,1) y (1,N), no se transforma la relación en una tabla independiente.

La clave primaria de casting (cod_casting) se propaga a fase como clave foránea, y junto al atributo parcial nro_secuencia, forma la clave primaria compuesta de fase.

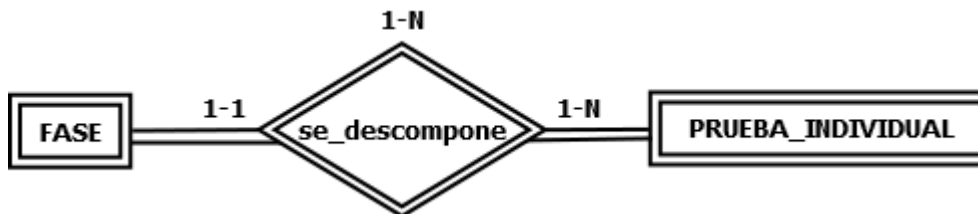
Los atributos de la relación (si los hubiera) también se incluirían en la tabla fase.

Como **resultado**, las tablas quedan de la siguiente forma:

CASTING (cod_casting, presupuesto, nombre, fecha_contratacion)

FASE (nro_secuencia, *cod_casting*, fecha_inicio)

4. Relación entidad FASE - se_descompone – entidad PRUEBA_INDIVIDUAL



*Figura 12 – relación fase **se descompone** prueba_individual transformación MER A MR*

Debido a que la entidad prueba_individual es una entidad débil por dependencia en identificación respecto de fase, y la relación se_descompone tiene cardinalidad (1,1) y (1,N), no se transforma la relación en una tabla independiente.

La clave primaria de fase (cod_casting, nro_secuencia) se propaga a prueba_individual como clave foránea, y junto con el atributo parcial nro_prueba, forma la clave primaria compuesta de prueba_individual. Los atributos de la relación (si los hubiera) también se incluirían en la tabla prueba_individual.

Como **resultado**, las tablas quedan de la siguiente forma:

FASE (**nro_secuencia**, **cod_casting**, fecha_inicio)

PRUEBA_INDIVIDUAL (**nro_secuencia**, **cod_casting**, **nro_prueba**, fecha_realizacion, hora_inicio, hora_finalizacion, sala, resultado)

5. Relación entidad CASTING – se asigna – entidad PERFIL

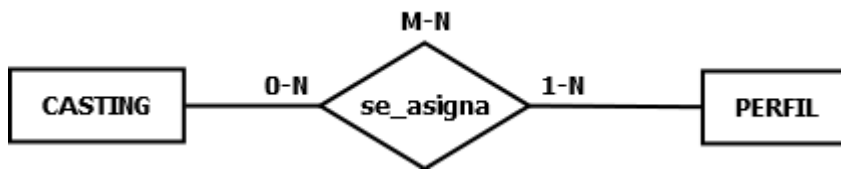


Figura 13 – relación casting *se asigna* perfil transformación MER A MR

Con una cardinalidad (M: N), esta relación sí se transforma en una tabla independiente y se crea la tabla **se_asigna** con las claves primarias de casting y perfil como clave primaria compuesta (**cod_casting**, **cod_perfil**).

Como **resultado**, las tablas quedan de la siguiente forma:

CASTING (**cod_casting**, presupuesto, nombre, fecha_contratacion)

SE_ASIGNA (**cod_casting**, **cod_perfil**)

PERFIL (**cod_perfil**, atributos)

6. Relación candidato – tiene – perfil

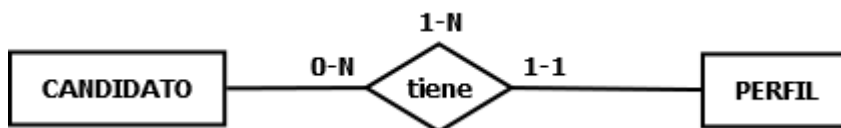


Figura 14 – relación candidato *tiene* perfil transformación MER A MR

Al tener cardinalidad (1,1) y (0, N), esta relación no se convierte en una tabla independiente y la clave primaria de perfil(**cod_perfil**) se propaga candidato como clave foránea.

Como **resultado**, las tablas quedan de la siguiente forma:

CANDIDATO (**cod_candidato**, nombre, dirección, **cod_perfil**)

PERFIL (**cod_perfil**, atributos)

7. Relación candidato – realiza – prueba individual

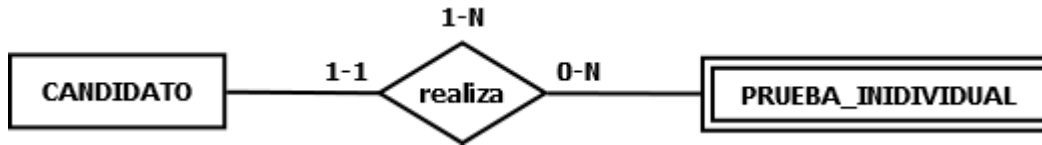


Figura 15 – relación candidato realiza prueba_individual transformación MER A MR

Debido a la cardinalidad (1,1) en candidato y (0,N) en prueba_individual, esta relación no se transforma en una tabla independiente. La clave primaria de candidato (cod_candidato) se propaga a la tabla prueba_individual como clave foránea.

Como resultado, las tablas quedan de la siguiente forma:

CANDIDATO (cod_candidato, nombre, dirección, cod_perfil)

PRUEBA_INDIVIDUAL (cod_casting, nro_secuencia, nro_prueba, fecha, hora_inicio, hora_fin, sala, resultado, cod_candidato)

2. Grafo relacional

El grafo relacional es la representación de un sistema mediante un conjunto de relaciones vinculadas entre sí por una o varias claves ajenas. Representa gráficamente el modelo relacional incluyendo entidades, atributos y restricciones.

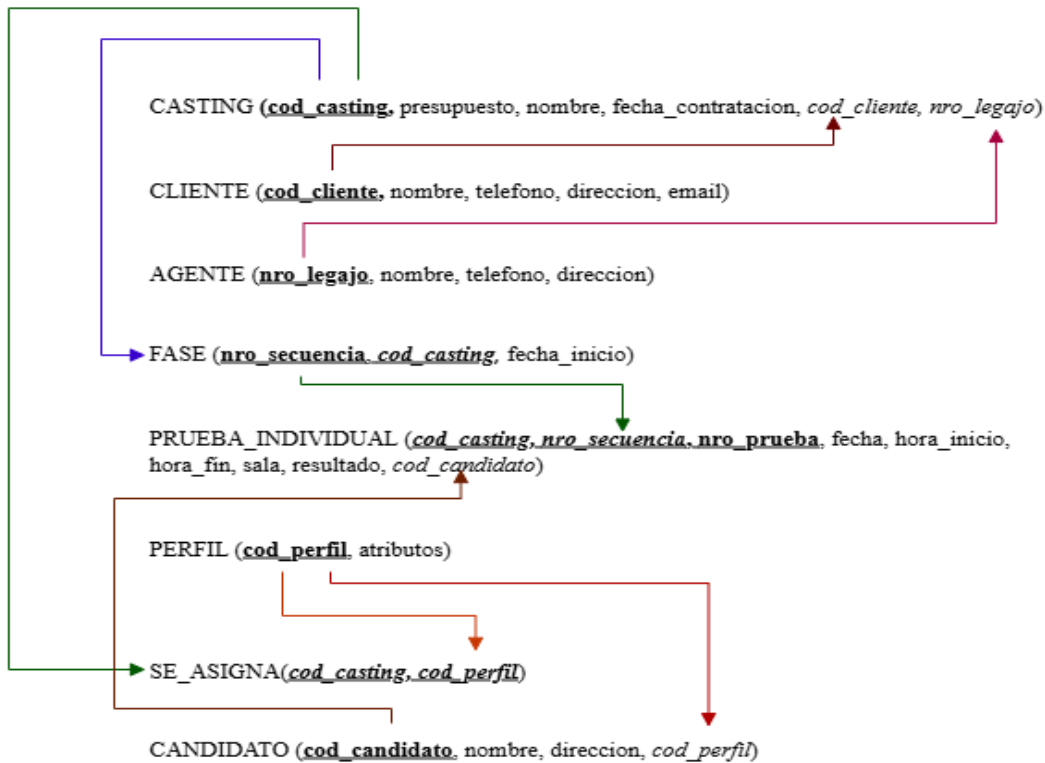


Figura 16 – grafo relacional

c) Normalización

La normalización es un proceso clave en el diseño de bases de datos relacionales, cuyo objetivo es estructurar las tablas de manera óptima. Esto se logra para:

- **Reducir la redundancia:** Evitar que la misma información se almacene repetidamente, optimizando el espacio y simplificando el mantenimiento de los datos.
- **Minimizar anomalías:** Prevenir inconsistencias y problemas que pueden surgir al insertar, actualizar o eliminar información en la base de datos.

Este proceso implica aplicar un conjunto de reglas o "formas normales" a las tablas. Cuanto más alto sea el nivel de normalización de una tabla, mejor será su diseño en términos de eficiencia y consistencia de los datos.

Para este trabajo final, nos centraremos en las primeras tres formas normales:

1. **Primera Forma Normal (1FN):** Una tabla cumple con la 1FN si cada columna contiene valores atómicos e indivisibles. Esto significa que cada celda de la tabla debe contener un único valor, sin listas o grupos repetitivos.
2. **Segunda Forma Normal (2FN):** Una tabla está en 2FN si ya cumple con la 1FN y todos sus atributos no clave (aquellos que no forman parte de la clave primaria) dependen completamente de la clave primaria. En el caso de claves primarias compuestas, ningún atributo no clave debe depender solo de una parte de esa clave.
3. **Tercera Forma Normal (3FN):** Una tabla está en 3FN si cumple con la 2FN y no presenta dependencias transitivas. Esto significa que ningún atributo no clave debe depender de otro atributo no clave; todos deben depender directamente de la clave primaria.

En este contexto, las dependencias funcionales juegan un papel central.

Una **dependencia funcional** describe una relación entre atributos de una tabla, donde un conjunto de atributos determina de forma única a otro. Se expresa como: $A \rightarrow B$

Esto significa que, si dos filas tienen el mismo valor en A, también deben tener el mismo valor en B. Es decir, B depende funcionalmente de A.

Las dependencias funcionales son clave para detectar cuándo una tabla está bien diseñada. De ellas se derivan los criterios de las formas normales:

- **Dependencia funcional completa:** cuando un atributo depende de toda la clave primaria y no de una parte de ella. Es esencial para alcanzar la 2FN.
- **Dependencia funcional parcial:** se da cuando un atributo depende solo de una parte de la clave primaria (en caso de claves compuestas). Se elimina al pasar de 1FN a 2FN.
- **Dependencia funcional transitiva:** se produce cuando un atributo depende de otro atributo que, a su vez, depende de la clave. Este tipo se elimina al alcanzar la 3FN.

Por lo tanto, comprender y aplicar correctamente las dependencias funcionales es esencial para garantizar un modelo de datos eficiente, coherente y libre de redundancias.

1. Identificación de los elementos necesarios para la normalización

- ❖ Para la tabla **CASTING**, la **PK** (clave primaria) es **cod_casting** y las dependencias funcionales serían:
cod_casting -> presupuesto, nombre, fecha_contratacion
- ❖ Para la tabla **CLIENTE**, la **PK** (clave primaria) es **cod_cliente** y las dependencias funcionales serían:
cod_cliente -> nombre, teléfono, dirección, email
- ❖ Para la tabla **AGENTE**, la **PK** (clave primaria) es **nro_legajo** y las dependencias funcionales serían:
nro_legajo -> nombre, teléfono, dirección
- ❖ Para la tabla **FASE**, la **PK** (clave primaria) es **nro_secuencia, cod_casting** y las dependencias funcionales serían:
nro_secuencia, cod_casting -> fecha_inicio
- ❖ Para la tabla **PRUEBA_INDIVIDUAL**, la **PK** (clave primaria) es **cod_casting, nro_secuencia, nro_prueba** y las dependencias funcionales serían:
cod_casting, nro_secuencia, nro_prueba -> fecha_realizacion, hora_inicio, hora_fin, sala, resultado
- ❖ Para la tabla **PERFIL**, la **PK** (clave primaria) es **cod_perfil** y las dependencias funcionales serían:
cod_perfil -> atributos
- ❖ Para la tabla **SE_ASIGNA**, la **PK** (clave primaria) es **cod_casting, cod_perfil** y las dependencias funcionales serían:
cod_casting, cod_perfil -> Ø
- ❖ Para la tabla **CANDIDATO**, la **PK** (clave primaria) es **cod_candidato** y las dependencias funcionales serían:
cod_candidato -> nombre, dirección

2. Identificación de formas normales del esquema inicial obtenido

1) Primera Forma Normal (1FN)

Entidad	Cumple/No cumple
CASTING	Cumple. Todos sus atributos tienen valores atómicos
CLIENTE	No cumple, el atributo direccion es un atributo compuesto, por ende, no todos sus atributos son atómicos.
AGENTE	No cumple, el atributo direccion es un atributo compuesto, por ende, no todos sus atributos son atómicos.

FASE	Cumple. Todos sus atributos tienen valores atómicos
PRUEBA_INDIVIDUAL	Cumple. Todos sus atributos tienen valores atómicos
PERFIL	Cumple. Todos sus atributos tienen valores atómicos
SE_ASIGNA	Cumple. Todos sus atributos tienen valores atómicos
CANDIDATO	No cumple, el atributo direccion es un atributo compuesto, por ende, no todos sus atributos son atómicos.

- ❖ Para la entidad **CLIENTE**, el atributo direccion es compuesto. Como en la Primera Forma Normal cada campo debe contener un único valor atómico, se descompone dirección en dos atributos simples: nro_casa, calle.
- ❖ Para la entidad **AGENTE**, el atributo direccion es compuesto. Como en la Primera Forma Normal cada campo debe contener un único valor atómico, se descompone dirección en dos atributos simples: nro_casa, calle.
- ❖ Para la entidad **CANDIDATO**, el atributo direccion es compuesto. Como en la Primera Forma Normal cada campo debe contener un único valor atómico, se descompone dirección en dos atributos simples: nro_casa, calle.

2) Segunda forma normal (2FN)

- ✓ **CASTING** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria(cod_casting).
- ✓ **CLIENTE** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria(cod_cliente).
- ✓ **AGENTE** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria(nro_legajo).
- ✓ **FASE** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria compuesta (nro_secuencia, cod_casting).
- ✓ **PRUEBA_INDIVIDUAL** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria compuesta (cod_casting, nro_secuencia, nro_prueba)
- ✓ **PERFIL** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria(cod_perfil).
- ✓ **SE_ASIGNA** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria.
- ✓ **CANDIDATO** -> Cumple ya que todos los atributos no clave dependen enteramente de la clave primaria(cod_candidato).

3. Normalizar a 3FN

- ✓ **CASTING:** Cumple ya que no existen dependencias entre atributos no clave. Todos dependen directamente de la clave primaria.
- ✓ **CLIENTE:** Cumple ya que no existen atributos no clave que dependan de otro atributo no clave.
- ✓ **AGENTE:** Cumple ya que no existen atributos no clave que dependan de otro atributo no clave.
- ✓ **FASE:** Cumple ya que no existen atributos no clave que dependan de otro atributo no clave.
- ✓ **PRUEBA_INDIVIDUAL:** Cumple ya que no todos los atributos no clave dependen directamente de la clave primaria compuesta y no entre sí.
- ✓ **PERFIL:** Cumple ya que todos los atributos dependen directamente de la clave.
- ✓ **SE_ASIGNA:** Cumple ya que no tiene atributos no clave, por lo que cumple automáticamente con 3FN.
- ✓ **CANDIDATO:** Cumple ya que no existen dependencias transitivas.

d) Diseño lógico específico

El diseño lógico específico es la representación del modelo lógico ajustada a las particularidades del SGBD que se va a utilizar. En esta etapa se definen:

- ✓ Los tipos de datos exactos según el SGBD (como VARCHAR, INT, DATE, etc.)
- ✓ Las claves primarias y foráneas
- ✓ Las restricciones de integridad sobre los atributos
- ✓ Reglas adicionales permitidas por el sistema (como valores por defecto, unicidad o nulos)

Este diseño prepara la base de datos para su implementación real, contemplando no solo la estructura lógica, sino también aspectos técnicos que afectan el rendimiento y la integridad de los datos.

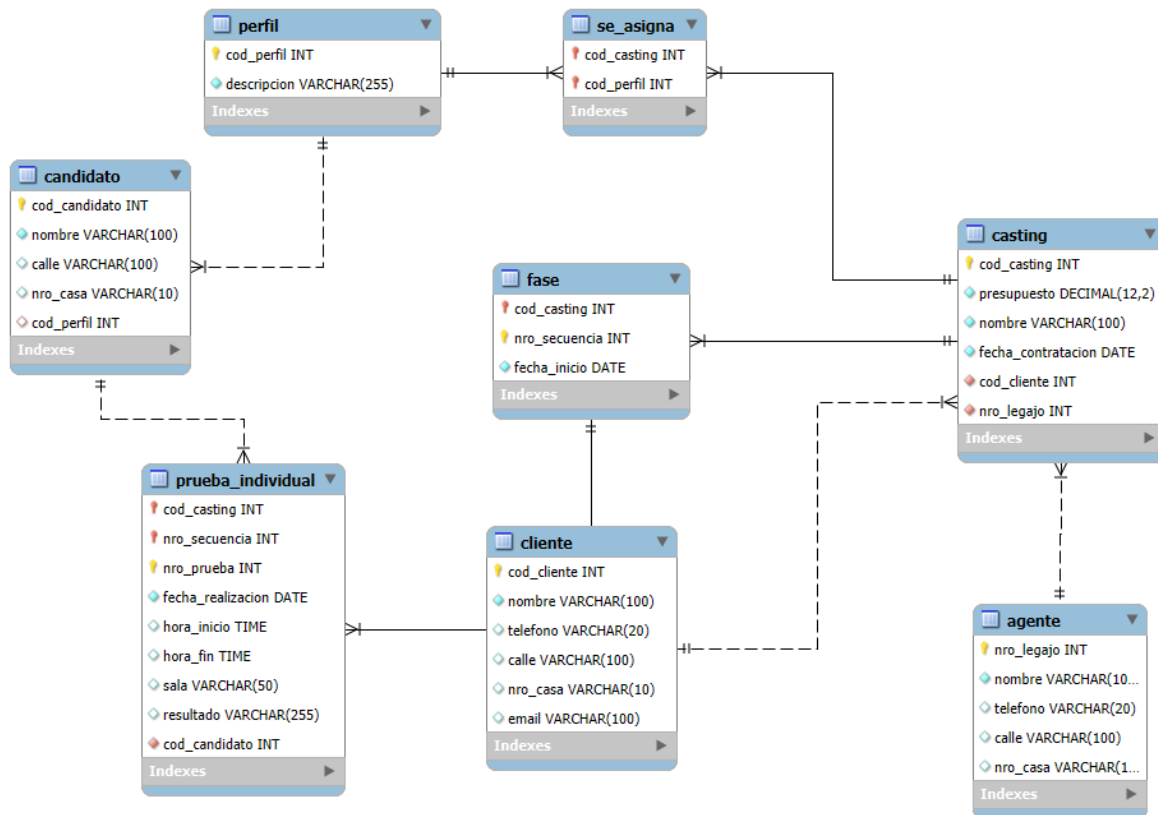


Figura 17 – modelo lógico específico

9. Implementación

Creación de base de datos

```
CREATE database IF NOT exists agencia_castings;
USE agencia_castings;
```

Creación de tablas

Tabla perfil

```
CREATE TABLE IF NOT EXISTS PERFIL (
    cod_perfil INT PRIMARY KEY,
    atributos VARCHAR(255)
);
```

Tabla cliente

```
CREATE TABLE IF NOT EXISTS CLIENTE (  
    cod_cliente INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    telefono VARCHAR(20),  
    email VARCHAR(100),  
    nro_casa VARCHAR(10),  
    calle VARCHAR(100)  
);
```

Tabla agente

```
CREATE TABLE IF NOT EXISTS AGENTE (  
    nro_legajo INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    telefono VARCHAR(20),  
    nro_casa VARCHAR(10),  
    calle VARCHAR(100)  
);
```

Tabla candidato

```
CREATE TABLE IF NOT EXISTS CANDIDATO (  
    cod_candidato INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    nro_casa VARCHAR(10),  
    calle VARCHAR(100),  
    cod_perfil INT,  
    FOREIGN KEY (cod_perfil) REFERENCES PERFIL(cod_perfil)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Tabla casting

```
CREATE TABLE IF NOT EXISTS CASTING (  
    cod_casting INT PRIMARY KEY,  
    nombre VARCHAR(100),  
    fecha_contratacion DATE,  
    presupuesto DECIMAL(10,2),  
    cod_cliente INT,  
    nro_legajo INT,  
    FOREIGN KEY (cod_cliente) REFERENCES CLIENTE(cod_cliente)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    FOREIGN KEY (nro_legajo) REFERENCES AGENTE(nro_legajo)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Tabla fase

```
CREATE TABLE IF NOT EXISTS FASE (  
    nro_secuencia INT,  
    cod_casting INT,  
    fecha_inicio DATE,  
    PRIMARY KEY (nro_secuencia, cod_casting),  
    FOREIGN KEY (cod_casting) REFERENCES CASTING(cod_casting)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```

Tabla se_asigna

```
CREATE TABLE IF NOT EXISTS SE_ASIGNA (  
    cod_casting INT,  
    cod_perfil INT,  
    PRIMARY KEY (cod_casting, cod_perfil),  
    FOREIGN KEY (cod_casting) REFERENCES CASTING(cod_casting)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (cod_perfil) REFERENCES PERFIL(cod_perfil)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

Tabla prueba_individual

```
CREATE TABLE IF NOT EXISTS PRUEBA_INDIVIDUAL (  
    cod_casting INT,  
    nro_secuencia INT,  
    nro_prueba INT,  
    fecha_realizacion DATE,  
    hora_inicio TIME,  
    hora_finalizacion TIME,  
    sala VARCHAR(50),  
    resultado ENUM('apto', 'no apto') DEFAULT NULL,  
    cod_candidato INT,  
    PRIMARY KEY (cod_casting, nro_secuencia, nro_prueba),  
    FOREIGN KEY (cod_casting, nro_secuencia) REFERENCES FASE(cod_casting, nro_secuencia)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (cod_candidato) REFERENCES CANDIDATO(cod_candidato)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT unique_sala_fecha_hora UNIQUE (sala, fecha_realizacion, hora_inicio, hora_finalizacion)  
);
```

Carga de datosTabla perfil

```
INSERT INTO PERFIL (cod_perfil, atributos) VALUES  
(1, 'Actor principal, 25-35 años, experiencia en drama'),  
(2, 'Modelo publicitario, altura > 1.75m'),  
(3, 'Actor secundario, 18-25 años, comedia'),  
(4, 'Actor de acción, 30-40 años, experiencia en escenas de riesgo'),  
(5, 'Modelo fotográfico, 20-30 años'),  
(6, 'Actor infantil, 8-14 años, experiencia en comedia'),  
(7, 'Cantante, 25-45 años, experiencia en musicales'),  
(8, 'Bailarín, 18-35 años, danza contemporánea'),  
(9, 'Actor de doblaje, experiencia en locución'),  
(10, 'Modelo de pasarela, altura > 1.80m'),  
(11, 'Actor de teatro, 25-50 años, experiencia en drama'),  
(12, 'Extra, 18-60 años, sin experiencia requerida');
```

Tabla cliente

```
INSERT INTO CLIENTE (cod_cliente, nombre, telefono, email, nro_casa, calle) VALUES
(1, 'Producciones Luna', '123456789', 'luna@prod.com', '123', 'Av. Libertad'),
(2, 'Estudio Solaris', '987654321', 'solaris@estudio.com', '456', 'Calle Principal'),
(3, 'Cine Estrella', '555123456', 'estrella@cine.com', '789', 'Av. Independencia'),
(4, 'Teatro Aurora', '321654987', 'aurora@teatro.com', '234', 'Calle del Sol'),
(5, 'Media Vision', '654987321', 'vision@media.com', '567', 'Av. Progreso'),
(6, 'Producciones Estelares', '147258369', 'estelares@prod.com', '890', 'Calle Luna'),
(7, 'Cine Nova', '258369147', 'nova@cine.com', '345', 'Av. Estrella'),
(8, 'Publicidad Orion', '369147258', 'orion@publi.com', '678', 'Calle Central'),
(9, 'Estudio Galaxia', '741852963', 'galaxia@estudio.com', '901', 'Av. Norte'),
(10, 'Cine Cosmos', '852963741', 'cosmos@cine.com', '1234', 'Calle Sur'),
(11, 'Producciones Zenith', '963741852', 'zenith@prod.com', '5678', 'Av. Oeste'),
(12, 'Teatro Eclipse', '159357246', 'eclipse@teatro.com', '9012', 'Calle Este');
```

Tabla agente

```
INSERT INTO AGENTE (nro_legajo, nombre, telefono, nro_casa, calle) VALUES
(101, 'Ana Gómez', '111222333', '101', 'Calle Sol'),
(102, 'Carlos Pérez', '444555666', '202', 'Av. Luna'),
(103, 'María López', '777888999', '303', 'Calle Estrella'),
(104, 'Luis Fernández', '222333444', '404', 'Av. Aurora'),
(105, 'Sofía Martínez', '555666777', '505', 'Calle Progreso'),
(106, 'Diego Ramírez', '888999111', '606', 'Av. Estela'),
(107, 'Clara Sánchez', '333444555', '707', 'Calle Nova'),
(108, 'Javier Torres', '666777888', '808', 'Av. Orion'),
(109, 'Laura Gómez', '999111222', '909', 'Calle Galaxia'),
(110, 'Pedro Díaz', '111333555', '1010', 'Av. Cosmos'),
(111, 'Elena Ruiz', '444666888', '1111', 'Calle Zenith'),
(112, 'Miguel Castro', '777999111', '1212', 'Av. Eclipse');
```

Tabla candidato

```
INSERT INTO CANDIDATO (cod_candidato, nombre, nro_casa, calle, cod_perfil) VALUES
(1001, 'Juan Martínez', '501', 'Calle Norte', 1),
(1002, 'Sofía Ramírez', '502', 'Av. Sur', 2),
(1003, 'Lucía Fernández', '503', 'Calle Oeste', 1),
(1004, 'Diego Torres', '504', 'Av. Este', 3),
(1005, 'Clara Sánchez', '505', 'Calle Central', 2),
(1006, 'Martín López', '506', 'Av. Aurora', 4),
(1007, 'Valeria Gómez', '507', 'Calle Progreso', 5),
(1008, 'Gabriel Díaz', '508', 'Av. Estela', 6),
(1009, 'Camila Ruiz', '509', 'Calle Nova', 7),
(1010, 'Lucas Fernández', '510', 'Av. Orion', 8),
(1011, 'Paula Castro', '511', 'Calle Galaxia', 9),
(1012, 'Rodrigo Pérez', '512', 'Av. Cosmos', 10);
```

Tabla casting

```
INSERT INTO CASTING (cod_casting, nombre, fecha_contratacion, presupuesto, cod_cliente, nro_legajo) VALUES
(100, 'Película de drama', '2025-06-01', 50000.00, 1, 101),
(101, 'Anuncio publicitario', '2025-07-01', 20000.00, 2, 102),
(102, 'Serie de comedia', '2025-07-15', 35000.00, 3, 103),
(103, 'Película de acción', '2025-08-01', 60000.00, 4, 104),
(104, 'Campaña fotográfica', '2025-08-15', 15000.00, 5, 105),
(105, 'Serie infantil', '2025-09-01', 30000.00, 6, 106),
(106, 'Musical teatral', '2025-09-15', 45000.00, 7, 107),
(107, 'Espectáculo de danza', '2025-10-01', 25000.00, 8, 108),
(108, 'Doblaje de película', '2025-10-15', 20000.00, 9, 109),
(109, 'Desfile de moda', '2025-11-01', 18000.00, 10, 110),
(110, 'Obra de teatro', '2025-11-15', 40000.00, 11, 111),
(111, 'Extra para comercial', '2025-12-01', 10000.00, 12, 112);
```

Tabla fase

```

INSERT INTO FASE (nro_secuencia, cod_casting, fecha_inicio) VALUES
(1, 100, '2025-07-20'), -- Fase 1: Película de drama
(2, 100, '2025-07-25'), -- Fase 2: Película de drama
(1, 101, '2025-07-22'), -- Fase 1: Anuncio publicitario
(1, 102, '2025-07-23'), -- Fase 1: Serie de comedia
(2, 102, '2025-07-28'), -- Fase 2: Serie de comedia
(1, 103, '2025-08-05'), -- Fase 1: Película de acción
(1, 104, '2025-08-20'), -- Fase 1: Campaña fotográfica
(1, 105, '2025-09-05'), -- Fase 1: Serie infantil
(1, 106, '2025-09-20'), -- Fase 1: Musical teatral
(1, 107, '2025-10-05'), -- Fase 1: Espectáculo de danza
(1, 108, '2025-10-20'), -- Fase 1: Doblaje de película
(1, 109, '2025-11-05'); -- Fase 1: Desfile de moda

```

Tabla se_asigna

```

INSERT INTO SE_ASIGNA (cod_casting, cod_perfil) VALUES
(100, 1), -- Película de drama: Actor principal
(100, 3), -- Película de drama: Actor secundario
(101, 2), -- Anuncio publicitario: Modelo
(101, 5), -- Anuncio publicitario: Modelo fotográfico
(102, 3), -- Serie de comedia: Actor secundario
(103, 4), -- Película de acción: Actor de acción
(104, 5), -- Campaña fotográfica: Modelo fotográfico
(105, 6), -- Serie infantil: Actor infantil
(106, 7), -- Musical teatral: Cantante
(107, 8), -- Espectáculo de danza: Bailarín
(108, 9), -- Doblaje de película: Actor de doblaje
(109, 10); -- Desfile de moda: Modelo de pasarela

```

Tabla prueba individual

```
INSERT INTO PRUEBA_INDIVIDUAL (cod_casting, nro_secuencia, nro_prueba, fecha_realizacion, hora_inicio, hora_finalizacion, sala, resultado, cod_candidato) VALUES
(100, 1, 1, '2025-07-20', '09:00:00', '10:00:00', 'Sala A', 'apto', 1001), -- Juan, Película de drama
(100, 1, 2, '2025-07-20', '10:30:00', '11:30:00', 'Sala A', 'no apto', 1003), -- Lucía, Película de drama
(100, 2, 1, '2025-07-25', '09:00:00', '10:00:00', 'Sala B', 'apto', 1001), -- Juan, Película de drama
(101, 1, 1, '2025-07-22', '14:00:00', '15:00:00', 'Sala C', 'apto', 1002), -- Sofía, Anuncio publicitario
(101, 1, 2, '2025-07-22', '15:30:00', '16:30:00', 'Sala C', 'apto', 1005), -- Clara, Anuncio publicitario
(102, 1, 1, '2025-07-23', '11:00:00', '12:00:00', 'Sala A', 'apto', 1004), -- Diego, Serie de comedia
(102, 2, 1, '2025-07-28', '09:00:00', '10:00:00', 'Sala B', NULL, 1004), -- Diego, Serie de comedia
(103, 1, 1, '2025-08-05', '09:00:00', '10:00:00', 'Sala D', 'apto', 1006), -- Martín, Película de acción
(104, 1, 1, '2025-08-20', '10:00:00', '11:00:00', 'Sala E', 'apto', 1007), -- Valeria, Campaña fotográfica
(105, 1, 1, '2025-09-05', '11:00:00', '12:00:00', 'Sala F', 'no apto', 1008), -- Gabriel, Serie infantil
(106, 1, 1, '2025-09-20', '12:00:00', '13:00:00', 'Sala G', 'apto', 1009), -- Camila, Musical teatral
(107, 1, 1, '2025-10-05', '13:00:00', '14:00:00', 'Sala H', NULL, 1010); -- Lucas, Espectáculo de danza
```

Consultas propuestas

- 1) Obtener todos los candidatos con la cantidad de castings en los que participaron

```
SELECT c.cod_candidato, c.nombre, c.nro_casa, c.calle, COUNT(DISTINCT
pi.cod_casting) AS cantidad_castings
FROM PRUEBA_INDIVIDUAL pi
JOIN CANDIDATO c ON pi.cod_candidato = c.cod_candidato
GROUP BY c.cod_candidato, c.nombre;
```

Resultado:

- ❖ Se utiliza COUNT(DISTINCT) para contar sin repetir los castings

	cod_candidato	nombre	nro_casa	calle	cantidad_castings
▶	1001	Juan Martínez	501	Calle Norte	1
	1002	Sofía Ramírez	502	Av. Sur	1
	1003	Lucía Fernández	503	Calle Oeste	1
	1004	Diego Torres	504	Av. Este	1
	1005	Clara Sánchez	505	Calle Central	1
	1006	Martín López	506	Av. Aurora	1
	1007	Valeria Gómez	507	Calle Progreso	1
	1008	Gabriel Díaz	508	Av. Estela	1
	1009	Camila Ruiz	509	Calle Nova	1
	1010	Lucas Fernández	510	Av. Orion	1

- ❖ Se utiliza GROUP BY cod_candidato para agrupar los datos por candidato

2) Obtener perfiles asignados a más de un casting distinto

```
SELECT p.cod_perfil, p.atributos, COUNT(DISTINCT sa.cod_casting) AS
cantidad_castings
FROM SE_ASIGNA sa
JOIN PERFIL p ON sa.cod_perfil = p.cod_perfil
GROUP BY p.cod_perfil, p.atributos
HAVING COUNT(DISTINCT sa.cod_casting) > 1;
```

Resultado:

	cod_perfil	atributos	cantidad_castings
▶	3	Actor secundario, 18-25 años, comedia	2
	5	Modelo fotográfico, 20-30 años	2

- ❖ Se utiliza DISTINCT dentro del COUNT () para evitar contar el mismo casting más de una vez.
- ❖ Se utiliza GROUP BY p.cod_perfil, p.nombre para agrupar por perfil para así contar cuántos castings tiene cada uno.
- ❖ HAVING COUNT (DISTINCT sa.cod_casting) > 1: filtra los perfiles que tienen más de un casting asignado.

3) Obtener los candidatos que tienen todas sus pruebas individuales con resultado “apto”

```
SELECT c.cod_candidato, c.nombre, c.nro_casa, c.calle
FROM CANDIDATO c
WHERE NOT EXISTS (
    SELECT 1
    FROM PRUEBA_INDIVIDUAL pi
    WHERE pi.cod_candidato = c.cod_candidato AND resultado = 'no apto'
);
```

Resultado:

	cod_candidato	nombre	nro_casa	calle
▶	1001	Juan Martínez	501	Calle Norte
	1002	Sofía Ramírez	502	Av. Sur
	1004	Diego Torres	504	Av. Este
	1005	Clara Sánchez	505	Calle Central
	1006	Martín López	506	Av. Aurora
	1007	Valeria Gómez	507	Calle Progreso
	1009	Camila Ruiz	509	Calle Nova
	1010	Lucas Fernández	510	Av. Orion
	1011	Paula Castro	511	Calle Galaxia
	1012	Rodrigo Pérez	512	Av. Cosmos

- ❖ Se utiliza NOT EXISTS para eliminar candidatos con pruebas individuales con resultado “no apto”

4) Obtener el casting con mayor cantidad de pruebas individuales realizadas

```
SELECT ca.cod_casting, ca.nombre,
ca.fecha_contratacion, ca.presupuesto, COUNT(*) AS total_pruebas
FROM CASTING ca
JOIN PRUEBA_INDIVIDUAL pi ON ca.cod_casting = pi.cod_casting
GROUP BY ca.cod_casting, ca.nombre
ORDER BY total_pruebas DESC
LIMIT 1;
```

Resultado:

	cod_casting	nombre	fecha_contratacion	presupuesto	total_pruebas
▶	100	Película de drama	2025-06-01	50000.00	3

- ❖ Se utiliza ORDER BY total_pruebas DESC para ordenar los resultados de mayor a menor según la cantidad total de pruebas, para que el casting con más pruebas quede primero.
- ❖ Se utiliza LIMIT 1 para limitar la salida Y así mostrar sólo un registro: el casting con la mayor cantidad de pruebas.

5) Obtener los castings que requieren más de un perfil para su realización

```
SELECT cod_casting, COUNT(cod_perfil) AS cantidad_perfiles
FROM SE_ASIGNA
GROUP BY cod_casting
HAVING COUNT(cod_perfil) > 1;
```

Resultado:

	cod_casting	cantidad_perfiles
▶	100	2
	101	2

- ❖ Se utiliza GROUP BY cod_casting para agrupar las filas por cada casting para así poder contar cuántos perfiles tiene asignados cada uno.
- ❖ Se utiliza HAVING COUNT(cod_perfil) > 1 para filtrar sólo aquellos castings que tienen más de un perfil asignado, descartando los que tienen uno o ninguno.

6) Obtener el perfil que ha sido asignado a la mayor cantidad de castings

```

SELECT p.cod_perfil, p.atributos, COUNT(*) AS
cantidad_asignaciones
FROM SE_ASIGNA sa
JOIN PERFIL p ON sa.cod_perfil = p.cod_perfil
GROUP BY p.cod_perfil, p.atributos
ORDER BY cantidad_asignaciones DESC
LIMIT 1;

```

Resultado:

	cod_perfil	atributos	cantidad_asignaciones
▶	3	Actor secundario, 18-25 años, comedia	2

- ❖ Se utiliza GROUP BY p.cod_perfil, p.atributos para agrupar los resultados por perfil para así contar cuántas asignaciones tiene cada uno.
- ❖ Se utiliza ORDER BY cantidad_asignaciones DESC para ordenar los perfiles de mayor a menor cantidad de asignaciones.
- ❖ Se utiliza LIMIT 1 para obtener sólo el perfil con la mayor cantidad de asignaciones.

7) Candidatos que participaron en el casting 101 pero no en el 100

```

SELECT ca.cod_candidato, ca.nombre, ca.nro_casa, ca.calle,
pi.fecha_realizacion, pi.hora_inicio, pi.hora_finalizacion, pi.sala,
pi.resultado
FROM CANDIDATO ca
JOIN PRUEBA_INDIVIDUAL pi ON ca.cod_candidato = pi.cod_candidato
WHERE pi.cod_casting = 101
AND NOT EXISTS (
    SELECT 1
    FROM PRUEBA_INDIVIDUAL pi2
    WHERE pi2.cod_candidato = ca.cod_candidato AND pi2.cod_casting =
100
);

```

Resultado:

	cod_candidato	nombre	nro_casa	calle	fecha_realizacion	hora_inicio	hora_finalizacion	sala	resultado
▶	1002	Sofía Ramírez	502	Av. Sur	2025-07-22	14:00:00	15:00:00	Sala C	apto
	1005	Clara Sánchez	505	Calle Central	2025-07-22	15:30:00	16:30:00	Sala C	apto

- ❖ Se utiliza WHERE pi.cod_casting = 101 para restringir la búsqueda a ese casting específico.
- ❖ Se utiliza NOT EXISTS para excluir a aquellos candidatos que también hayan participado en el casting 100.

- ❖ La subconsulta busca si existe al menos una prueba de ese candidato en el casting 100. Si no la encuentra, el candidato queda en el resultado.

8) Obtener agentes asignados a castings de alto presupuesto y múltiples fases

```
SELECT a.nro_legajo, a.nombre, a.telefono, a.nro_casa, a.calle,
COUNT(DISTINCT c.cod_casting) AS total_castings, SUM(c.presupuesto)
AS total_presupuesto
FROM AGENTE a
JOIN CASTING c ON a.nro_legajo = c.nro_legajo
WHERE c.cod_casting IN (
    SELECT cod_casting
    FROM FASE
    GROUP BY cod_casting
    HAVING COUNT(*) > 1
)
AND c.presupuesto > (
    SELECT AVG(presupuesto)
    FROM CASTING
)
GROUP BY a.nro_legajo, a.nombre, a.telefono, a.nro_casa, a.calle
HAVING total_castings >= 1;
```

Resultado:

	nro_legajo	nombre	telefono	nro_casa	calle	total_castings	castings_gestionados	total_presupuesto
▶	101	Ana Gómez	111222333	101	Calle Sol	1	100	50000.00
	103	María López	777888999	303	Calle Estrella	1	102	35000.00

- ❖ Se utiliza una subconsulta en la cláusula WHERE con GROUP BY y HAVING COUNT(*) > 1 para seleccionar aquellos castings que tienen más de una fase.
- ❖ Se utiliza otra subconsulta para calcular el promedio de presupuestos (AVG) y comparar si el presupuesto del casting es superior.
- ❖ Se utiliza COUNT(DISTINCT c.cod_casting) para contar la cantidad de castings diferentes que cumple las condiciones por cada agente.
- ❖ Se utiliza SUM(c.presupuesto) para sumar los presupuestos de los castings filtrados por agente.
- ❖ Se agrupa por todos los datos del agente con GROUP BY, y se filtra el resultado final con HAVING para mostrar solo aquellos que hayan dirigido al menos un casting bajo esas condiciones.

- 9) Obtener los candidatos con resultado “apto” en pruebas individuales de los castings que tienen más fases

```

SELECT ca.cod_candidato, ca.nombre, ca.nro_casa, ca.calle,
COUNT(*) AS pruebas_aprobadas
FROM CANDIDATO ca
JOIN PRUEBA_INDIVIDUAL pi ON ca.cod_candidato = pi.cod_candidato
WHERE pi.resultado = 'apto'
AND pi.cod_casting IN (
    SELECT cod_casting
    FROM FASE
    GROUP BY cod_casting
    HAVING COUNT(*) = (
        SELECT MAX(num_fases)
        FROM (
            SELECT COUNT(*) AS num_fases
            FROM FASE
            GROUP BY cod_casting
        ) AS sub
    )
)
GROUP BY ca.cod_candidato, ca.nombre, ca.nro_casa, ca.calle;

```

Resultado:

	cod_candidato	nombre	nro_casa	calle	pruebas_aprobadas
▶	1001	Juan Martínez	501	Calle Norte	2
	1004	Diego Torres	504	Av. Este	1

- ❖ Se filtra con WHERE aquellas pruebas que hayan sido aprobadas (resultado = 'apto') y que pertenezcan a los castings con la máxima cantidad de fases. Para identificar esos castings, se utiliza una subconsulta anidada:
 - Primero, se agrupan las fases por cod_casting y se cuentan (COUNT (*)) para saber cuántas fases tiene cada casting.
 - Luego, se obtiene el valor máximo de fases con MAX(num_fases) sobre ese resultado.
 - Finalmente, se compara en el HAVING para traer solo los castings con esa cantidad máxima de fases.
- ❖ Se utiliza GROUP BY para agrupar los resultados por cada candidato, y se usa COUNT(pi.nro_prueba) para saber cuántas pruebas aprobó.

- 10) Obtener los candidatos que hayan participado en pruebas en castings donde el presupuesto es mayor a 30.000 y al promedio de dichos castings, excluyendo a quienes aprobaron en castings de menor presupuesto.

```

SELECT ca.cod_candidato, ca.nombre, ca.nro_casa, ca.calle,
       COUNT(*) AS pruebas_aprobadas,
       SUM(c.presupuesto) AS total_presupuesto_castings
FROM CANDIDATO ca
JOIN PRUEBA_INDIVIDUAL pi ON ca.cod_candidato = pi.cod_candidato
JOIN CASTING c ON pi.cod_casting = c.cod_casting
WHERE pi.resultado = 'apto'
AND c.presupuesto > 30000
AND c.presupuesto > (
    SELECT AVG(presupuesto)
    FROM CASTING
    WHERE presupuesto > 30000
)
AND NOT EXISTS (
    SELECT 1
    FROM PRUEBA_INDIVIDUAL pi2
    JOIN CASTING c2 ON pi2.cod_casting = c2.cod_casting
    WHERE pi2.cod_candidato = ca.cod_candidato
    AND pi2.resultado = 'apto'
    AND c2.presupuesto <= 30000
)
GROUP BY ca.cod_candidato, ca.nombre, ca.nro_casa, ca.calle
HAVING pruebas_aprobadas >= 1
ORDER BY pruebas_aprobadas DESC, ca.nombre;

```

Resultado:

	cod_candidato	nombre	nro_casa	calle	pruebas_aprobadas	total_presupuesto_castings
▶	1001	Juan Martínez	501	Calle Norte	2	100000.00
	1006	Martín López	506	Av. Aurora	1	60000.00

- ❖ Se utiliza la cláusula WHERE para:
 - Filtrar solo las pruebas con resultado 'apto'.
 - Incluir castings con presupuesto mayor a 30.000.
 - Verificar además que dicho presupuesto sea superior al promedio de castings mayores a 30.000.
- ❖ Se utiliza NOT EXISTS para asegurarse de que el candidato no haya aprobado ninguna prueba en castings con presupuesto igual o menor a 30.000.
- ❖ Se utiliza GROUP BY para agrupar los resultados por cada candidato, y se agregan funciones como:
 - COUNT (pi.nro_prueba) para obtener la cantidad de pruebas aprobadas.

- SUM(c.presupuesto) para calcular el total de presupuesto de los castings en los que participó.
- ❖ Se utiliza HAVING para excluir candidatos que no hayan aprobado ninguna prueba y ORDER BY para ordenar primero por cantidad de pruebas y luego alfabéticamente por nombre.

10. Conclusión

En este trabajo integrador se diseñó e implementó una base de datos relacional con el objetivo de poder satisfacer los requerimientos solicitados por una agencia de castings ficticia. Esto se hizo a través de seis etapas: el análisis y recopilación de requisitos, el diseño conceptual, la elección del SGBD, el diseño lógico, el diseño físico y por último la implementación. Cada etapa permitió aplicar los contenidos teóricos vistos durante la cursada, como la normalización, el modelo entidad-relación y las dependencias funcionales.

Al comenzar con el trabajo, interpretar y traducir correctamente la información del caso a una estructura de base de datos fue un proceso complejo. A medida que avanzaba el desarrollo del trabajo, los conceptos vistos en la cursada comenzaron a tomar forma dentro del modelo, lo que permitió representar las relaciones y restricciones de manera coherente.

En conclusión, este trabajo permitió aplicar lo aprendido durante la cursada a través de un caso concreto. Además, puso en evidencia la importancia estructurar con claridad cada etapa del diseño e implementación de una base de datos, ya que diseñar una base de datos con una estructura clara no solo facilita la implementación, sino que también permite mantener la coherencia y optimizar el manejo de la información en entornos reales.

11.Bibliografía

1. [EN 2002] Elmasri, R.; Navathe, S.B.: Fundamentos de Sistemas de Bases de Datos. 3^a Ed. Addison-Wesley.
2. [MPM 1999] de Miguel, A.; Piattini, M.; Marcos, E.: Diseño de bases de datos relacionales.
3. Apuntes de cátedra – Bases de Datos 2022